

RMSProp (Root Mean Squared Propagation)

Optimizers in Deep Learning – Part 5

1. What is RMSProp?

RMSProp (Root Mean Squared Propagation) is an improvement over **AdaGrad**.

Remember how **AdaGrad**'s learning rate kept getting smaller and smaller?

Eventually, it became **too tiny**, and training almost stopped.

RMSProp fixes that problem by **slowing down the decay** of the learning rate — using an **exponentially decaying average** of past squared gradients instead of adding all of them up forever.

In simple words:

- **AdaGrad** remembers *all* past gradients → learning rate keeps shrinking.
- **RMSProp** remembers *recent* gradients more → learning rate stays fresh and adaptive.

This helps the model keep learning efficiently throughout training.

2. Why RMSProp Was Needed

Let's quickly recap the problem with AdaGrad:

$$G_t = G_{t-1} + g_t^2$$

→ The sum G_t keeps increasing forever

→ Denominator $\sqrt{G_t}$ keeps increasing

→ Learning rate $\frac{\eta}{\sqrt{G_t}}$ keeps decreasing

→ Model stops learning too early.

- RMSProp fixes this by replacing the **sum** with a **decaying average** of past gradients.
-

3. RMSProp Formula (Step-by-Step)

Let's understand what happens mathematically

Step **1**: Compute the exponentially weighted moving average of squared gradients

$$E[g^2]_t = \beta E[g^2]_{t-1} + (1 - \beta) g_t^2$$

Here:

- $E[g^2]_t$: moving average of squared gradients
 - g_t : gradient at time t
 - β : decay rate (commonly 0.9)
 - This makes recent gradients more important, older ones fade away.
-

Step **2**: Update the parameters

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{E[g^2]_t + \epsilon}} g_t$$

Where:

- η : learning rate
 - ϵ : small number (like 1e-8) to avoid division by zero 
-

So, what's happening?

- The denominator $\sqrt{E[g^2]_t}$ acts like a **normalizer**.
- If recent gradients are large $\rightarrow E[g^2]_t$ is large $\rightarrow$ learning rate becomes smaller.
- If gradients are small $\rightarrow$ learning rate becomes larger.

This balances the step size for each parameter dynamically.

4. Visual Intuition (with Animation or Imagination)

Imagine you're trying to roll a ball down a **bumpy valley** (the loss function).

- If you roll too fast on a steep slope, you'll overshoot.
- If you move too slow, you'll take forever to reach the bottom.

RMSProp helps by watching **how steep the path has been recently**, and automatically adjusting the speed (learning rate).

It **keeps movement smooth and controlled**, avoiding both overshooting and stalling.

5. The Math Behind RMSProp (Simplified)

RMSProp stands for **Root Mean Square of the Gradients Propagation**.

Let's expand it:

1 Compute moving average of squared gradients

→ “Mean Squared”

2 Take the square root

→ “Root Mean Squared”

3 Use that value to normalize the learning rate (Propagation step)

That's literally why it's called **RMSProp**.

6. Typical Hyperparameter Values

Parameter	Typical Value	Description
Learning Rate (η)	0.001	Often smaller than SGD
β (Decay Rate)	0.9	Controls how fast old gradients fade
ϵ	1e-8	Prevents division by zero

7. Advantages of RMSProp

1. Keeps learning rate adaptive not shrinking too much like AdaGrad
2. Works very well for **non-stationary** and **online** learning problems (like RNNs)
3. Handles noisy gradients efficiently (useful in deep networks)
4. Converges faster than standard SGD

8. Disadvantages

1. Needs tuning of hyperparameters (β and η)
2. May still overshoot if learning rate is too high
3. Sensitive to initialization in some architectures
4. However, when used properly, RMSProp works extremely well and is used in **most modern deep learning tasks**, often as a base for more advanced optimizers like **Adam**.

9. Implementation (Keras / TensorFlow)

```
from tensorflow.keras.optimizers import RMSprop
```

```
optimizer = RMSprop(learning_rate=0.001, rho=0.9, epsilon=1e-8)
```

Here:

- rho = decay rate (β)
- RMSProp automatically manages the decaying average of squared gradients.

10. Comparison - AdaGrad vs RMSProp

Feature	AdaGrad	RMSProp
Gradient Memory	Accumulates all past gradients	Uses decaying average
Learning Rate	Keeps decreasing forever	Stabilizes over time
Works Well For	Sparse data	Deep networks, RNNs
Problem	Learning halts early	Needs tuning of β
Performance	Slower convergence	Faster, smoother convergence

11. Summary - In One Look

Concept	RMSProp Summary
Full Name	Root Mean Squared Propagation
Key Idea	Use a decaying average of squared gradients
Solves Problem	Learning rate decay in AdaGrad
Behavior	Smooths out learning with recent gradient history
Best For	Deep Networks, RNNs, CNNs
Main Hyperparameters	η (learning rate), β (decay rate), ϵ

12. Intuitive Analogy

1. Imagine you're walking down a hill in fog.
2. You can't see far, but you remember the steepness of the recent steps.
3. You adjust your pace slower when it's steep, faster when it's flat.
4. That's exactly how RMSProp behaves *it remembers recent terrain and adapts accordingly*.

13. Takeaways

1. RMSProp is AdaGrad with a **memory fade** older gradients slowly lose influence.
2. Prevents learning from stopping too early.
3. Balances step sizes automatically for each parameter.
4. Forms the foundation for more advanced optimizers like **Adam** (which combines RMSProp + Momentum).

TL;DR:

RMSProp keeps an exponentially decaying memory of past gradients to maintain a stable and adaptive learning rate allowing faster and smoother convergence than AdaGrad.